

BEGINNER-FRIENDLY GUIDE

YOLO26, Transfer Learning, Fine-Tuning & Incremental Learning

A complete guide from zero, in easy English, with practical code you can run.

YOLO architecture

What is new in YOLO26

Transfer Learning

Fine-Tuning

Incremental Learning

Export & deploy

Table of Contents

1. Basics you need first
2. What is Object Detection?
3. YOLO Architecture: how it works inside
4. Setup and preparing your dataset
5. Practical 1: Predictions with a pretrained YOLO
6. Transfer Learning
7. Fine-Tuning
8. Incremental Learning (Continual Learning)
9. Evaluation, Export and Deployment
10. Common mistakes and fixes
11. Cheat sheet and learning roadmap

Note: This version of the guide uses **YOLO26** (`yolo26n.pt` , `yolo26s.pt` and so on) with the **Ultralytics** library. The code is the same for YOLOv8, YOLO11 and YOLO26. Only the weights file name changes (for example `yolo26s.pt` becomes `yolo11s.pt`). Section 3 explains the classic YOLO design (YOLOv8 style). **Section 3.10 explains what is new in YOLO26** (no NMS, no DFL, new optimizer). Always update Ultralytics first: `pip install -U ultralytics` .

1. Basics You Need First

1.1 Neural Network and CNN

- A **neural network** is a stack of layers. An image goes in, and a prediction comes out. During training, the network changes its **weights** to make fewer mistakes.
- A **CNN (Convolutional Neural Network)** is a network made for images. Small **filters** slide over the image and find patterns.
- Early layers find edges, lines and colors.
- Middle layers find textures and shapes.
- Last layers find object parts (eye, wheel, hand).

This layer-by-layer idea is the **main reason Transfer Learning works** (see section 6).

1.2 Bounding Box

A rectangle around an object. There are two common formats:

Format	Values	Used for
xyxy	x_min, y_min, x_max, y_max	Drawing, evaluation
xywh (YOLO format)	x_center, y_center, width, height (normalized 0 to 1)	YOLO label files

1.3 IoU (Intersection over Union)

IoU tells how much two boxes overlap.

$$\text{IoU} = \text{Area of Overlap} / \text{Area of Union}$$

- IoU = 1 means a perfect match. IoU = 0 means no overlap.
- A detection is usually called "correct" when **IoU is 0.5 or more**.

1.4 NMS (Non-Maximum Suppression)

The model often draws many boxes on one object. NMS cleans this up:

1. Sort all boxes by confidence.
2. Keep the box with the highest confidence.
3. Remove other boxes that overlap it too much (IoU above a threshold).
4. Repeat for the remaining boxes.

1.5 Precision, Recall, AP, mAP

- **Precision:** Of all detections the model made, how many were correct? $\text{TP} / (\text{TP} + \text{FP})$
- **Recall:** Of all real objects, how many did the model find? $\text{TP} / (\text{TP} + \text{FN})$
- **AP (Average Precision):** The area under the Precision-Recall curve for one class.
- **mAP:** The average of AP over all classes.
- **mAP50:** measured at IoU 0.5.
- **mAP50-95:** average from IoU 0.5 to 0.95. It is stricter and more reliable.

1.6 Train / Validation / Test split

- **Train** (about 70 to 80%): the model learns from this.
- **Validation** (about 10 to 20%): used to check progress during training.
- **Test** (about 10%): used once at the end for the final score.

The same image must never be in both train and validation. If it is, your score will look better than it really is (this is called data leakage).

2. What is Object Detection?

Task	Question	Output
Classification	What is in this image?	One label
Detection	What is it, and where ?	Boxes + class + confidence
Segmentation	Which pixels belong to which object?	Masks

Two types of detectors

Two-stage (R-CNN family): first find possible regions, then classify each region. Accurate, but slow.

One-stage (YOLO, SSD): look at the whole image **once** and predict boxes and classes together. **YOLO means "You Only Look Once"**. It is great for real-time use.

A short history of YOLO

Version	Main change
YOLOv1 (2016)	Split the image into a grid and detect in one pass
YOLOv2 / v3	Anchor boxes, predictions at 3 scales
YOLOv4 / v5	CSP backbone, mosaic augmentation, PyTorch code
YOLOv8 (Ultralytics)	Anchor-free , decoupled head, C2f module, Task-Aligned assigner
YOLO11	Better speed and accuracy, same workflow
YOLO26	NMS-free (end-to-end) inference, DFL removed , MuSGD optimizer, built for fast CPU and edge use

3. YOLO Architecture: How It Works Inside

How to read this section: Sections 3.1 to 3.9 explain the classic YOLO design (YOLOv8 style). This is the base you must understand first. YOLO26 keeps the same big picture (Backbone, Neck, Head) but changes a few important parts. These changes are listed in **Section 3.10**, so read it right after 3.9.

Every modern YOLO has **three parts**:

```
Input Image (640 x 640 x 3)
  |
  v
+-----+
|  BACKBONE  | -> Finds features (edges -> shapes -> objects)
+-----+
  | (feature maps at 3 scales: P3, P4, P5)
  v
+-----+
|   NECK     | -> Mixes the features (FPN + PAN)
+-----+
  |
  v
+-----+
|   HEAD     | -> Predicts box + class
+-----+
  |
  v
Raw predictions -> NMS -> Final detections
```

3.1 Backbone (Feature Extractor)

Job: pull useful features out of the image.

The YOLOv8 backbone is **CSPDarknet** style:

- **Conv layers:** with stride 2, they make the image smaller and add more channels.
- **C2f module:** splits features into two paths and joins them again. This helps gradients flow and saves computation.
- **SPPF (Spatial Pyramid Pooling Fast):** at the end, it looks at the image with different pooling sizes to understand context. This helps with both small and big objects.

The backbone gives **3 feature maps**: | Level | Stride | Size for 640 input | Best for | |---|---|---|---|
| P3 | 8 | 80 × 80 | **Small** objects | | P4 | 16 | 40 × 40 | Medium objects | | P5 | 32 | 20 × 20 | **Big** objects |

3.2 Neck (Feature Fusion): FPN + PAN

- **FPN (Feature Pyramid Network):** goes top-down. It carries "what is it" information from deep layers down to bigger maps.
- **PAN (Path Aggregation Network):** goes bottom-up. It carries "where is it" information from early layers up.

Result: each scale gets both "what" and "where" information.

3.3 Head (Prediction)

The YOLOv8 head is **decoupled** and **anchor-free**.

Decoupled: two separate branches.

- **Box branch:** predicts the box location (regression).
- **Class branch:** predicts the class probability.

(Older YOLO versions used one shared branch. Splitting them improved accuracy.)

Anchor-free: older models predicted small changes to fixed "anchor boxes". Now the model directly predicts the distance from the object center to the box edges. You no longer need to tune anchors.

3.4 Output Tensor Shape

For a 640×640 input:

```
Number of predictions =  $80 \times 80 + 40 \times 40 + 20 \times 20 = 6400 + 1600 + 400 = 8400$ 
```

For COCO (80 classes) the output shape is **(1, 84, 8400)**.

- $84 = 4$ (box: x, y, w, h) + 80 (class scores)
- 8400 = total candidate predictions

These go through a confidence filter and NMS to give the final detections.

For your own 3 classes, the output is **(1, 7, 8400)** ($4 + 3$).

3.5 Model Sizes

Model	Params (approx.)	Speed	Accuracy	When to use
<code>n</code> (nano)	about 3M	Fastest	Lower	Mobile, edge devices, Raspberry Pi
<code>s</code> (small)	about 11M	Fast	OK	Balanced
<code>m</code> (medium)	about 26M	Medium	Good	General use
<code>l</code> (large)	about 44M	Slow	Very good	Server GPU
<code>x</code> (extra large)	about 68M	Slowest	Best	Maximum accuracy

The parameter counts above are approximate **YOLOv8** numbers, used to show the idea. YOLO26 also comes in `n`, `s`, `m`, `l`, `x`. For exact YOLO26 sizes and scores, see the Ultralytics YOLO26 docs. As a reference point, the official docs report **YOLO26n = 40.9 mAP** and **YOLO26x = 57.5 mAP** on COCO.

Tip: start with `n` or `s`. When the pipeline works, try a bigger model.

3.6 Loss Function (How the Model Learns)

The total loss has three parts:

1. **Box loss (CIoU):** the difference between the predicted and real box. CIoU also looks at center distance and aspect ratio, not only overlap.
2. **DFL (Distribution Focal Loss):** predicts each box edge as a distribution, which makes edges more precise.
3. **Class loss (BCE):** the error in class predictions.

In the training log you will see `box_loss`, `cls_loss` and `df_l_loss`. **If all losses go down, the model is learning.**

YOLO26 note: YOLO26 **removes the DFL module** (see 3.10). Its training log may show different loss names than the classic ones above. The rule stays the same: the losses should go down, and validation mAP should go up.

3.7 Label Assignment (TAL)

During training, each real object must be matched to some of the model's predictions. YOLOv8 uses the **Task-Aligned Assigner**. It picks predictions that have both a good class score and a good box (IoU).

3.8 Key Training Techniques

- **Mosaic augmentation:** joins 4 images into one. The model sees different scenes and scales. It is turned off in the last epochs (`close_mosaic=10`).
- **HSV color change, flip, scale, translate:** adds variety to the data.
- **Pretrained weights:** a model already trained on the COCO dataset.

3.9 Inference Pipeline (How One Image Is Processed)

```
Image -> Resize and pad (letterbox 640x640) -> Normalize (0 to 1)
      -> Backbone -> Neck -> Head -> 8400 predictions
      -> Confidence filter (conf=0.25) -> NMS (iou=0.7)
      -> Final boxes (in original image size)
```

This is the classic pipeline. In YOLO26 the NMS step is **not needed** (see 3.10).

3.10 What Is New in YOLO26

YOLO26 is the newest Ultralytics model family covered in this guide. It is built to be **faster on CPU and edge devices** and **simpler to deploy**. The main changes are:

1. NMS-free, end-to-end inference

- **Classic YOLO:** the model makes many overlapping boxes for each object. Then NMS (a separate step) removes the extra boxes.
- **YOLO26:** the model is trained with **one-to-one matching**. This means each object gets **one prediction**, so duplicates are removed at the source. The Ultralytics docs describe an **optional one-to-one detection head** that gives predictions **without NMS**.

Why this matters:

- The exported model is **self-contained**. The output is already the final list of detections.
- You do **not** need to write or tune an NMS step on each platform (CPU, mobile, TensorRT, and so on).
- Post-processing is faster, which helps most on small CPU devices.

```
Classic YOLO : Image -> Model -> many boxes -> NMS -> final boxes
YOLO26       : Image -> Model -> final boxes
```

Because the output format is different from classic YOLO, the output tensor shape in 3.4 (`1, 84, 8400`) **does not apply to YOLO26 end-to-end mode**. If you write your own post-processing code for an exported model, **print the output shape first** and check the Ultralytics export docs. The official sources used for this guide do not state the exact shape, so no number is given here.

2. DFL removed

YOLO26 removes the **Distribution Focal Loss (DFL)** module. This makes the detection head **simpler** and easier to export, while still giving accurate boxes. (In classic YOLOv8, DFL helped make box edges precise, see 3.6.)

3. MuSGD optimizer

A **hybrid optimizer** that mixes **SGD** with **Muon**. The goal is better and steadier training. You do not need to code anything for this; it is part of how YOLO26 is trained.

4. ProgLoss

Progressive Loss Balancing. It slowly shifts the training focus toward the head that is used at inference time. This makes training more stable and improves accuracy.

5. STAL (Small-Target-Aware Label Assignment)

An improved way to match real objects with predictions, so **small objects** are not missed. Useful for drones, CCTV, far-away objects and similar cases.

6. Edge-first speed

The official docs report up to **43% faster CPU inference (ONNX)** for YOLO26n compared with YOLO11n, and about **1.7 ms on a T4 GPU with TensorRT** for the nano model. It exports to **TensorRT, ONNX, CoreML, TFLite and OpenVINO**, with **FP16 and INT8** quantization support.

7. One family, many tasks

YOLO26 supports detection, instance segmentation, semantic segmentation, depth estimation, classification, pose estimation and oriented boxes (OBB). The weight names follow this pattern:

```
yolo26n.pt      # detection
yolo26n-seg.pt  # instance segmentation
yolo26n-cls.pt  # classification
yolo26n-pose.pt # pose estimation
yolo26n-obb.pt  # oriented boxes
```

What does NOT change for you

- The code you write is the **same**: `YOLO("yolo26s.pt")`, `.train()`, `.val()`, `.predict()`, `.export()`.
- The dataset format, `data.yaml`, labels and folders are the **same**.
- Transfer Learning, Fine-Tuning and Incremental Learning work in the **same way** (Sections 6, 7 and 8).

Quick comparison

	YOLOv8	YOLO26
Needs NMS at inference	Yes	No (end-to-end)
DFL in the head	Yes	Removed
Box matching	Many predictions per object	One-to-one
Optimizer	SGD / AdamW	MuSGD (hybrid of SGD + Muon)
Best for	General use	Edge, CPU, simple deployment

Sources for this section: the Ultralytics YOLO26 docs and blog posts (links at the end of the guide). Features may change as the library updates, so check docs.ultralytics.com for the latest details.

4. Setup and Preparing Your Dataset

4.1 Installation

```
# Python 3.8 or newer is needed. A virtual environment is recommended.
python -m venv yolo_env
source yolo_env/bin/activate      # Windows: yolo_env\Scripts\activate

pip install ultralytics
```

For a GPU, first install PyTorch for your CUDA version (get the command from pytorch.org), then install ultralytics.

Check your setup:

```
import torch
from ultralytics import YOLO
print("GPU available:", torch.cuda.is_available())
```

If you have no GPU, use Google Colab (free GPU).

4.2 Collecting Data

- Start with at least **100 to 200 images per class**. For good results, aim for **about 1500 images per class**.
- Add variety: different light, angles, backgrounds, distances and cameras.
- Add some **background images** (images with no object), about 0 to 10%. They reduce false detections.

4.3 Annotation (Labeling)

Free and popular tools: **Roboflow**, **CVAT**, **LabelImg**, **Label Studio**.

Each image gets one `.txt` file. Each object is one line:

```
<class_id> <x_center> <y_center> <width> <height>
```

All values are between **0 and 1** (divided by image width or height). Example:

```
0 0.512 0.430 0.200 0.350
1 0.250 0.700 0.120 0.180
```

4.4 Folder Structure

```
datasets/safety/  
├─ images/  
│   ├─ train/  img001.jpg, img002.jpg ...  
│   └─ val/    img501.jpg ...  
└─ labels/  
    ├─ train/  img001.txt, img002.txt ...  
    └─ val/    img501.txt ...
```

Ultralytics finds the label file by replacing `images` with `labels` in the path. So the file names must match (`img001.jpg` and `img001.txt`).

4.5 data.yaml

```
path: /content/datasets/safety    # dataset root (use an absolute path)  
train: images/train  
val: images/val  
  
names:  
  0: helmet  
  1: vest
```

4.6 Data Quality Checklist

- ☐ Every class ID in the label files exists in `names`
- ☐ No box goes outside the image
- ☐ No duplicate or near-duplicate images between train and val (for video frames, split by video)
- ☐ Boxes are tight, and **every visible object is labeled** (missing labels are the biggest problem)

5. Practical 1: Predictions with a Pretrained YOLO

First, see what a COCO-trained model can do with no training.

```
from ultralytics import YOLO

# Weights download automatically the first time
model = YOLO("yolo26n.pt")

# Single image
results = model.predict("bus.jpg", conf=0.25, save=True)

for r in results:
    print("Classes:", r.names)          # {0: 'person', 1: 'bicycle', ...}
    for box in r.bboxes:
        cls_id = int(box.cls[0])
        conf = float(box.conf[0])
        x1, y1, x2, y2 = box.xyxy[0].tolist()
        print(f"{r.names[cls_id]}  conf={conf:.2f}  box=({x1:.0f},{y1:.0f},{x2:.0f},
{y2:.0f})")
```

The output is saved in `runs/detect/predict/`.

Video and webcam:

```
model.predict("video.mp4", save=True)    # video file
model.predict(source=0, show=True)       # webcam
```

Problem: this model only knows the 80 COCO classes (person, car, dog, and so on). If you want your own classes like "helmet", "vest", "crack" or "weed", you need **Transfer Learning**. That is the next section.

6. Transfer Learning

6.1 What Is It?

Transfer Learning means using knowledge from one task to help with another similar task.

Simple example: if you know how to play cricket, learning baseball is easier. You already know how to hold a bat, watch the ball and run. You do not start from zero.

In YOLO: the COCO dataset has about 118,000 training images and 80 classes. A model trained on it has already learned edges, shapes, textures and object parts. These features **also help in your helmet and vest detection**. You only adapt this "brain" to your own data.

6.2 Why Use It?

Training from scratch	Transfer Learning
Needs lakhs of images	A few thousand (or even hundreds) is enough
Takes days or weeks	Takes minutes or hours
High risk of overfitting	Lower risk
Needs an expensive GPU	A small GPU is fine

6.3 What Gets "Transferred"?

Backbone (early layers)	-> general features	-> REUSE
Neck	-> feature mixing	-> reuse
Head (last class layer)	-> task-specific	-> START NEW (new classes)

When you give new classes (for example 2 classes), the **shape of the last class-prediction layer changes**. So that layer starts fresh. All other layers load from the pretrained weights. At the start of training you will see this line:

```
Transferred 349/355 items from pretrained weights
```

It means most layers are pretrained, and only a few (the head's class layers) are new. This is normal.

6.4 Two Ways

Way A: Feature Extraction (Freeze the Backbone)

Freeze the backbone weights and train only the neck and head.

- Use when: you have **very little data**, or your data looks like COCO.
- Good: fast, low overfitting risk.
- Bad: limited accuracy (the features cannot change).

Way B: Full Transfer Learning (Train All Layers)

Start from pretrained weights and **train all layers**.

- Use when: you have a fair amount of data (a few thousand images).
- This is the **default** in Ultralytics.

6.5 Practical 2: Transfer Learning on Your Data

Step 1: Load a pretrained model

```
from ultralytics import YOLO
model = YOLO("yolo26s.pt")      # pretrained COCO weights
```

Step 2: Train (default: full transfer learning)

```
results = model.train(
    data="datasets/safety/data.yaml",
    epochs=50,
    imgsz=640,
    batch=16,          # if GPU memory is low use 8 or 4; -1 = auto
    patience=15,        # stop if no improvement for 15 epochs (early stopping)
    project="runs/safety",
    name="transfer_v1",
    device=0,           # GPU. For CPU use device="cpu"
    seed=42,
)
```

Step 3: Way A with a frozen backbone

```
model = YOLO("yolo26s.pt")
model.train(
    data="datasets/safety/data.yaml",
    epochs=30,
    imgsz=640,
    freeze=10,          # freeze the first 10 layers (the backbone)
    name="transfer_frozen",
)
```

`freeze=10` means the first 10 layers (layers 0 to 9, mostly the backbone) will not be trained.

Check for YOLO26: the exact number of backbone layers can be different between model versions. The value 10 is the usual choice for YOLOv8 and works as a good starting point for newer versions too. To be sure, look at the model structure with `model.info(detailed=True)` and pick the layer number where the backbone ends.

Step 4: Look at the results

Folder: `runs/safety/transfer_v1/`

```
weights/best.pt      <- the best model (use this one)
weights/last.pt      <- the model from the last epoch (used to resume training)
results.csv          <- numbers for every epoch
results.png          <- loss and metric graphs
confusion_matrix.png <- which classes get confused
PR_curve.png         <- precision-recall curve
val_batch0_pred.jpg  <- predictions on validation images
```

Step 5: Validate and predict

```
best = YOLO("runs/safety/transfer_v1/weights/best.pt")

metrics = best.val(data="datasets/safety/data.yaml")
print("mAP50      :", metrics.box.map50)
print("mAP50-95  :", metrics.box.map)
print("Precision:", metrics.box.mp)
print("Recall    :", metrics.box.mr)

best.predict("test_images/", conf=0.4, save=True)
```

6.6 How to Read the Training Graphs

Item	Good sign	Bad sign
train/box_loss	Keeps going down	Stuck or going up
val/box_loss	Keeps going down	Train loss goes down but val loss goes up = Overfitting
mAP50-95	Keeps going up	Flat (more epochs will not help)
Confusion matrix	Dark diagonal	Two classes mixed up

6.7 Resume Training

If training stops (Colab disconnects, power cut):

```
model = YOLO("runs/safety/transfer_v1/weights/last.pt")
model.train(resume=True)
```

7. Fine-Tuning

7.1 What Is It? How Is It Different from Transfer Learning?

The two are closely related, and people often use the words as if they mean the same thing. Here is the clear difference:

- **Transfer Learning** is the big idea: "use a pretrained model for a new task."
- **Fine-Tuning** is one **way** of doing Transfer Learning: you take the pretrained weights and **adjust them slowly with a small learning rate**. The model fits your new data, but does not lose the good knowledge it already has.

	Feature Extraction	Fine-Tuning	From scratch
Pretrained weights	Yes	Yes	No
Backbone	Frozen	Trainable (low LR)	Trainable
Learning rate	Normal	Small	Normal or large
Data needed	Little	Medium	Very large
Risk	Underfitting	Overfitting / forgetting	Overfitting

7.2 Why Is Fine-Tuning Needed?

Imagine the pretrained model has seen mostly bright daytime photos, and your data is night CCTV footage (a different **domain**). Training only the head is not enough. The backbone also needs a small adjustment for your domain. Fine-tuning does this.

7.3 Think Layer by Layer

Early layers	-> general (edges, colors)	-> change little / freeze
Middle layers	-> medium features	-> change a little
Last layers	-> task-specific	-> change the most

7.4 Recommended Strategy: Two-Stage Fine-Tuning

Stage 1: Warm-up (backbone frozen). Let the new head learn first. If all layers train together while the head is random, big wrong gradients can damage the good pretrained backbone.

Stage 2: Unfreeze and use a small LR. Now fine-tune the whole model slowly.

Practical 3: Two-Stage Fine-Tuning

```
from ultralytics import YOLO

DATA = "datasets/safety/data.yaml"

# ----- Stage 1: Backbone frozen, new head learns -----
model = YOLO("yolo26s.pt")
model.train(
    data=DATA,
    epochs=20,
    imgsz=640,
    freeze=10,          # backbone frozen
    optimizer="SGD",
    lr0=0.01,           # the head is new, so a normal LR is fine
    project="runs/safety",
    name="ft_stage1",
)

# ----- Stage 2: Unfreeze everything, small LR -----
model = YOLO("runs/safety/ft_stage1/weights/best.pt") # best of stage 1
model.train(
    data=DATA,
    epochs=40,
    imgsz=640,
    freeze=0,          # nothing frozen
    optimizer="SGD",
    lr0=0.001,         # 10 times smaller LR <- the key idea of fine-tuning
    lrf=0.01,          # final LR = lr0 x lrf
    warmup_epochs=1,
    patience=15,
    project="runs/safety",
    name="ft_stage2",
)
```

Important tip: In Ultralytics, `optimizer` defaults to `"auto"`. In this mode, your `lr0` value is **ignored** (the library picks the LR itself). If you want control over the LR, always write `optimizer="SGD"` or `"AdamW"` clearly, as done above.

7.5 Important Hyperparameters

Parameter	Meaning	Advice for fine-tuning
<code>epochs</code>	How many times the model sees all the data	30 to 100 (with patience)
<code>batch</code>	Images per step	Based on your GPU (16 or 32)
<code>imgsz</code>	Image size	640 default; 960 or 1280 for small objects
<code>lr0</code>	Starting learning rate	SGD: 0.001 to 0.01, AdamW: 0.0001 to 0.001
<code>lrf</code>	Ratio for the final LR	0.01
<code>freeze</code>	Number of early layers frozen	0 or 10
<code>patience</code>	Early stopping	10 to 20
<code>weight_decay</code>	Regularization	0.0005 (default)
<code>mosaic</code> , <code>flip_lr</code> , <code>hsv_*</code>	Augmentation	Keep them on if data is small
<code>close_mosaic</code>	Turn off mosaic for the last N epochs	10

7.6 Rules of Thumb for Fine-Tuning

1. **Very little data (below about 500 images):** freeze the backbone (`freeze=10`), use a small model (`n` or `s`), and use more augmentation.
2. **A fair amount of data (a few thousand):** use two-stage fine-tuning.
3. **Very different domain (medical, satellite, X-ray):** unfreeze everything, train more epochs, and also tune `imgsz`.
4. **Keep the LR small for the pretrained part.** A big LR can erase pretrained knowledge.
5. **Change one thing at a time** and give each experiment its own `name` so you can compare.

7.7 Spot and Fix Overfitting

Sign: train loss goes down, but validation mAP goes down or stays flat.

Fixes:

- More data and more variety
- More augmentation
- Use `freeze` or a smaller model
- Early stopping with `patience`
- Slightly increase `weight_decay`
- Check label quality (this is often the real problem)

7.8 Compare Experiments

```
import pandas as pd

for run in ["transfer_v1", "transfer_frozen", "ft_stage2"]:
    df = pd.read_csv(f"runs/safety/{run}/results.csv")
    df.columns = df.columns.str.strip()
    best_map = df["metrics/mAP50-95(B)"].max()
    print(f"{run:20s} best mAP50-95 = {best_map:.3f}")
```

8. Incremental Learning (Continual Learning)

8.1 What Is the Problem?

You trained a model that finds **helmet** and **vest**. Now your client says: *"Please detect gloves too."*

The simplest way is to train the model only on the new gloves data. **Result:** the model learns gloves, but **forgets helmet and vest**.

This is called **Catastrophic Forgetting**. Learning new things erases old knowledge, because the weights change to fit the new data.

The goal of Incremental Learning: learn the new thing **and** remember the old things, without training the whole model from zero every time.

8.2 Types of Incremental Learning

Type	Meaning	Example
Class-incremental	New classes are added	Helmet, vest → + gloves
Domain-incremental	Same classes, new conditions	Day images → + night images
Task-incremental	New task	Detection → + segmentation

In this guide we do the most common one: **class-incremental object detection**.

8.3 A Special Problem in Detection: "Missing Labels"

This is very important. Many people get stuck here.

Say you labeled 500 new images, and you annotated **only gloves**. But those images also contain **helmets and vests that are not labeled**. During training, the model treats them as **"background"** and learns: *"when I see a helmet, say nothing."* This is the biggest cause of forgetting.

Fix: use the old model to create **pseudo-labels** (automatic labels) for helmet and vest on these new images, and add them.

8.4 Overview of Strategies

Strategy	How it works	Ease	Result
1. Replay (Rehearsal)	Mix a part of the old data with the new data and train	Easy	Very good
2. Pseudo-labeling / Distillation	The old model labels old classes on new images	Easy	Very good
3. Freezing	Freeze the backbone and train only the head	Very easy	OK, limited
4. Regularization (EWC, LwF)	Penalize changes to important weights	Hard	Research level
5. Parameter isolation / Adapters	Separate small modules for each task	Hard	Good

Advice for production: use **Replay + Pseudo-labeling + a low LR** together. It is the most practical and reliable option, and it works with Ultralytics without research code.

8.5 Practical 4: Class-Incremental Detection (Full Workflow)

Scenario

- **Base model (v1):** classes are 0: helmet , 1: vest (already trained)
- **New task:** add 2: gloves
- **New data:** in new_data/ , only gloves are labeled (class id 2)
- **Old data:** datasets/safety/ (original train and val)

Final folder plan

```
project/
├─ datasets/safety/          # old data (classes 0, 1)
├─ new_data/
│   ├─ images/train/ , images/val/  # new images
│   └─ labels/train/ , labels/val/  # only gloves (class 2)
├─ merged/                  # we will create this
│   ├─ images/{train,val}
│   └─ labels/{train,val}
└─ runs/safety/base_v1/weights/best.pt  # old model
```

Step 1: Save the old model's baseline score

You will need this later to measure forgetting.

```
from ultralytics import YOLO

old_model = YOLO("runs/safety/base_v1/weights/best.pt")
m_old = old_model.val(data="datasets/safety/data.yaml")

print("OLD model mAP50-95:", m_old.box.map)
for c in m_old.box.ap_class_index:
    print(f" {m_old.names[c]:10s} {m_old.box.maps[c]:.3f}")
```

Step 2: Make pseudo-labels for old classes on the new images

merge_data.py :

```
import shutil
from pathlib import Path
from ultralytics import YOLO

old_model = YOLO("runs/safety/base_v1/weights/best.pt")

PSEUDO_CONF = 0.5      # keep only confident predictions
NEW_CLASS_ID = 2       # gloves

def build_split(split):
    new_img_dir = Path(f"new_data/images/{split}")
    new_lbl_dir = Path(f"new_data/labels/{split}")
    out_img = Path(f"merged/images/{split}"); out_img.mkdir(parents=True, exist_ok=True)
    out_lbl = Path(f"merged/labels/{split}"); out_lbl.mkdir(parents=True, exist_ok=True)

    for img_path in sorted(new_img_dir.glob("*.jpg")):
        # (a) labels for old classes, made by the old model
        r = old_model.predict(img_path, conf=PSEUDO_CONF, verbose=False)[0]
        lines = []
        for cls, (x, y, w, h) in zip(r.boxes.cls.tolist(), r.boxes.xywhn.tolist()):
            lines.append(f"{int(cls)} {x:.6f} {y:.6f} {w:.6f} {h:.6f}")

        # (b) your own hand-made labels (gloves)
        manual = new_lbl_dir / f"{img_path.stem}.txt"
        if manual.exists():
            lines += [l for l in manual.read_text().strip().splitlines() if l.strip()]

        shutil.copy(img_path, out_img / img_path.name)
        (out_lbl / f"{img_path.stem}.txt").write_text("\n".join(lines))

    for s in ["train", "val"]:
        build_split(s)
    print("New data + pseudo-labels are ready.")
```

Pseudo-labels are not perfect. Start with `PSEUDO_CONF=0.5`. Open 20 to 50 images (in Roboflow or CVAT with the boxes drawn) and check that the boxes look right.

Step 3: Replay (add a part of the old data)

```
import random, shutil
from pathlib import Path

REPLAY_RATIO = 0.3      # 30% of the old data (or about the same amount as the new data)
random.seed(42)

def add_replay(split):
    old_imgs = sorted(Path(f"datasets/safety/images/{split}").glob("*.jpg"))
    k = int(len(old_imgs) * (REPLAY_RATIO if split == "train" else 1.0)) # val: all old val
    chosen = random.sample(old_imgs, k) if split == "train" else old_imgs

    for p in chosen:
        lbl = Path(f"datasets/safety/labels/{split}/{p.stem}.txt")
        # add a prefix so file names do not clash
        shutil.copy(p, f"merged/images/{split}/old_{p.name}")
        if lbl.exists():
            shutil.copy(lbl, f"merged/labels/{split}/old_{p.stem}.txt")

for s in ["train", "val"]:
    add_replay(s)
```

How much replay? More replay means better memory but longer training. Start with 20 to 50%. If old-class accuracy still drops, increase it.

Step 4: New `merged.yaml`

```
path: /content/project/merged
train: images/train
val: images/val

names:
  0: helmet
  1: vest
  2: gloves
```

Very important: keep the **old class IDs the same** (0, 1). Add the new class only **at the end** (2). If you change the order, everything the model learned about the old classes becomes useless.

Step 5: Train from the old weights with a small LR

```
from ultralytics import YOLO

model = YOLO("runs/safety/base_v1/weights/best.pt") # start from the old model
model.train(
    data="merged/merged.yaml",
    epochs=40,
    imgsz=640,
    optimizer="SGD",
    lr0=0.002,          # small LR so old knowledge is not disturbed much
    lrf=0.01,
    warmup_epochs=1,
    patience=15,
    project="runs/safety",
    name="incremental_v2",
)
```

At the start of training, you may see a message that some layers were not transferred. **This is normal.** The last class-prediction layer changed from 2 to 3 classes, so it starts fresh. The old classes are learned back from the replay data and the pseudo-labels.

(Advanced: you can also copy the old weights into the first two "rows" of the new class layer by hand. This is model surgery and is not required. The method above is enough in most cases.)

Step 6: Measure forgetting (the most important step)

```
new_model = YOLO("runs/safety/incremental_v2/weights/best.pt")
m_new = new_model.val(data="merged/merged.yaml")

print("NEW model mAP50-95:", m_new.box.map)
for c in m_new.box.ap_class_index:
    print(f"  {m_new.names[c]:10s} {m_new.box.maps[c]:.3f}")
```

Now compare:

Class	Old model (v1)	New model (v2)	Meaning
helmet	0.78	0.77	No real forgetting ✓
vest	0.74	0.72	Small drop, OK ✓
gloves	-	0.65	New class learned ✓

(These numbers are only examples. Your own data will give different numbers.)

Decision:

- Old-class mAP dropped **a lot** (for example 5 points or more)? Increase the replay ratio, lower the LR, and check the pseudo-label quality.
- New class is weak? Add more and better data for it, or train more epochs.

8.6 Strategy 3: With Freezing (Simplest, Less Good)

If you have no time for pseudo-labeling:

```
model = YOLO("runs/safety/base_v1/weights/best.pt")
model.train(data="merged/merged.yaml", epochs=30, freeze=10, lr0=0.002, optimizer="SGD",
            name="incremental_frozen")
```

The backbone stays frozen, so features are kept. But the head can still forget if labels are missing.
Without replay or pseudo-labels, this is not a complete solution.

8.7 Another Direct Way: Full Retrain

Whenever a new class comes, **merge all old + new data (with correct labels)** and train again from pretrained COCO weights. This is the **cleanest and most accurate** way.

- If your data is not huge and training takes only a few hours, **try this first**.
- Incremental learning helps when: the old data is not available (privacy or storage), training is very expensive, or you need small updates again and again.

Honestly: in production, "merge all data and fine-tune" often gives the most stable result. Choose incremental methods only when you really need them.

8.8 Domain-Incremental Example (Day to Night)

The classes are the same; only the conditions are new. **Replay** is the most useful tool here:

```
# merged data = old day images (replay) + new night images (all labeled)
model = YOLO("runs/safety/base_v1/weights/best.pt")
model.train(data="merged_night/data.yaml", epochs=30, optimizer="SGD", lr0=0.002,
            name="domain_inc_v2")
```

The classes did not change, so the head keeps the same shape and all layers load from the old model.

8.9 Rules for Incremental Learning

1. **Never change old class IDs.** Always add new classes at the end.
2. **Do not leave old-class objects unlabeled** in the new data (use pseudo-labels or label by hand).

3. **Use a small LR** (equal to or a little smaller than fine-tuning).
 4. **Always keep a replay buffer**, and update it a little after each incremental step.
 5. After each step, **evaluate on all old and new classes**, not only the new one.
 6. Save the weights and the data version of each step separately (v1, v2, v3) so you can go back.
 7. After many incremental steps (10 or more), drift builds up. Sometimes do a **full retrain** to "reset."
-

9. Evaluation, Export and Deployment

9.1 Correct Evaluation

- Always do the final check on a **separate test set** that was never used in training.
- Look at **per-class** numbers, not only overall mAP. One class can be weak while the average looks fine.
- In the confusion matrix, see which classes get mixed with the background or with other classes.
- Also test manually on real images (real camera, real lighting).

9.2 Choosing the Confidence Threshold

```
model.predict("img.jpg", conf=0.25)    # more detections, more false alarms
model.predict("img.jpg", conf=0.6)     # fewer detections, more certain
```

Use `PR_curve.png` and `F1_curve.png` to pick the `conf` where F1 is highest.

9.3 Export

```
model = YOLO("runs/safety/incremental_v2/weights/best.pt")

model.export(format="onnx")              # works on many platforms
model.export(format="engine", half=True) # NVIDIA TensorRT (GPU)
model.export(format="tflite")            # Android / edge devices
model.export(format="openvino")          # Intel CPU
```

After export you can still load it like `YOLO("best.onnx").predict(...)`.

9.4 Measure Speed

```
model.val(data="merged/merged.yaml", batch=1) # the val output shows ms per image
```

For real-time use, 25 to 30 FPS (about 30 to 40 ms per image) is usually enough.

10. Common Mistakes and Fixes

Problem	Cause	Fix
Very low mAP	Wrong or missing labels	Re-check labels; open 50 samples of each class
Good on train, bad on val	Overfitting / leakage	More data, more augmentation, fix the split
CUDA out of memory	Batch or imgsz too big	Lower <code>batch</code> , use <code>imgsz=480/512</code> , smaller model
"No labels found" at start	Folder or file name mismatch	Check <code>images/</code> and <code>labels/</code> structure and file names
Wrong class IDs in output	Order in <code>data.yaml</code>	Make the <code>names</code> order match the IDs in the label files
Small objects not found	Low resolution	<code>imgsz=960/1280</code> , or tiling (SAHI)
Many boxes on one object	NMS threshold (classic YOLO)	Lower <code>iou</code> (for example <code>iou=0.5</code>). YOLO26 in end-to-end mode does not use NMS, so also check your confidence value and that you use the latest Ultralytics
Old classes gone (incremental)	Catastrophic forgetting	Replay + pseudo-labels + small LR
<code>lr0</code> has no effect	<code>optimizer="auto"</code>	Write <code>optimizer="SGD"</code> or <code>"AdamW"</code> clearly
Loss is NaN	LR too big / bad labels	Lower LR, check labels (negative or above 1 values)
Colab disconnects	Session timeout	Use <code>resume=True</code> , save results to Google Drive

11. Cheat Sheet and Learning Roadmap

11.1 How to Decide: Which Method?

Are your classes already in COCO (person, car, dog...)?

- | - Yes -> Use the pretrained model directly (Section 5)
- | - No -> Label your own data

|

How much data do you have?

- | - Below 500 images -> Transfer Learning + freeze=10, small model
- | - 500 to 5000 -> Two-stage Fine-Tuning (Section 7)
- | - Above 5000 -> Full fine-tuning, try a bigger model

Later, a new class or domain comes?

- | - Old data is available and training is cheap -> Merge and fine-tune again (cleanest)
- | - No old data, or updates are frequent -> Incremental (Replay + Pseudo-labels)
(Section 8)

11.2 All Commands in One Place

```
from ultralytics import YOLO

# 1. Pretrained inference
YOLO("yolo26n.pt").predict("img.jpg", save=True)

# 2. Transfer learning
YOLO("yolo26s.pt").train(data="data.yaml", epochs=50, imgsz=640)

# 3. Frozen backbone
YOLO("yolo26s.pt").train(data="data.yaml", epochs=30, freeze=10)

# 4. Fine-tuning (small LR)
YOLO("best.pt").train(data="data.yaml", epochs=40, freeze=0,
                      optimizer="SGD", lr0=0.001)

# 5. Incremental (merged data + old weights)
YOLO("base_v1/weights/best.pt").train(data="merged.yaml", epochs=40,
                                       optimizer="SGD", lr0=0.002)

# 6. Evaluate / Export / Resume
YOLO("best.pt").val(data="data.yaml")
YOLO("best.pt").export(format="onnx")
YOLO("last.pt").train(resume=True)
```

11.3 Short Glossary

Word	Meaning
Backbone	The part that pulls features out of the image
Neck	The part that mixes features from different scales
Head	The final prediction part (box + class)
Anchor-free	Detection without fixed starting boxes
NMS	Removing duplicate boxes
IoU	Overlap of two boxes
mAP	Overall accuracy of detection
Transfer Learning	Reusing a model that already learned something
Fine-Tuning	Adjusting pretrained weights with a small LR
Freeze	Stopping some layers from being updated
Catastrophic Forgetting	Forgetting old knowledge when learning new things
Replay	Mixing a copy of old data into new training
Pseudo-label	A label made automatically by a model

11.4 Your Learning Roadmap (Week by Week)

Week	What to do	Goal
1	Python, PyTorch basics, CNN idea	Foundation
2	Run the Section 5 code, predict with the COCO model	Get comfortable with YOLO
3	Label your own small dataset (2 to 3 classes, 200+ images)	Data pipeline
4	Compare Transfer Learning and freeze (Section 6)	Your first custom model
5	Two-stage fine-tuning and hyperparameter tests (Section 7)	Understand tuning
6	Add a new class with the incremental workflow (Section 8)	See forgetting yourself and fix it
7	Export (ONNX), measure FPS, webcam or video demo	Deployment
8	Mini project: full end-to-end (data → train → evaluate → deploy)	Portfolio

11.5 Further Reading

- Ultralytics docs (docs.ultralytics.com): Train, Val and Export modes
- YOLO papers: YOLOv1 (Redmon et al.), YOLOv3, YOLOv4, and the YOLOv8 docs and blog
- YOLO26 (sources used for Section 3.10):

- Ultralytics YOLO26 docs: <https://docs.ultralytics.com/models/yolo26/>
 - Why YOLO26 removes NMS: <https://www.ultralytics.com/blog/why-ultralytics-yolo26-removes-nms-and-how-that-changes-deployment>
 - Meet YOLO26: <https://www.ultralytics.com/blog/meet-ultralytics-yolo26-a-better-faster-smaller-yolo-model>
 - Continual Learning: "Learning without Forgetting" (LwF), "Elastic Weight Consolidation" (EWC)
 - Practice datasets: free public datasets on Roboflow Universe
-

Final Advice from a Senior Engineer

1. **Data quality is more important than model size.** Better labels often help more than a bigger model.
2. **Start small.** Run the pipeline end to end first, then improve it.
3. **Record every experiment** (data version, hyperparameters, mAP).
4. **Do not trust only the overall mAP.** Also check each class and real-world images.
5. **The biggest mistake in incremental learning:** leaving old classes unlabeled in the new data. Always handle it with pseudo-labels or replay.